Rangkumin Setup Guide

Document version: 1.0 · September 7, 2026

This guide explains two ways to run Rangkumin: **locally for development** and **self-hosted on Cloudflare for production**. Rangkumin is designed for exactly two users and stores IDR monetary values as integers.

> Never place tokens, private keys, user email addresses, receipt images, or real transaction data in the repository, issues, screenshots, or public logs.

1. Architecture overview

Component	Implementation
Frontend	React + Vite, built into `public/`
API	TypeScript + Hono on Cloudflare Workers
Database	Cloudflare D1, `DB` binding
Production login	Cloudflare Access, with JWT verification repeated by the Worker
Receipt Scan	Workers AI, `AI` binding
Notifications	Dashboard + VAPID Web Push
PWA	Service worker, IndexedDB snapshot and offline outbox
Scheduler	Reminder/budget/push cron every 15 minutes and daily purge

Google Sheets and Telegram are not part of the active release. They are only optional candidates for future add-ons or updates. D1 remains the source of truth.

2. Requirements

- Node.js 22 or newer.
- npm and Git.
- For production: a Cloudflare account with Workers, D1, Workers AI, Zero Trust/Access, and an active Cloudflare-managed domain.
- Two distinct user email addresses.
- Chrome, Edge, or Chromium to regenerate the PDF documentation.

Verify local tools:

```
node --version
npm --version
git --version
```

3. Local setup

3.1 Get the source and dependencies

```
git clone https://github.com/andikafadil28/rangkumin.git
cd rangkumin
npm ci
```

3.2 Prepare the local database

Wrangler keeps local D1 state under `.wrangler/`. A remote Cloudflare database is not required for the basic local workflow.

```
npm run db:migrations:list:local
npm run db:migrate:local
npm run db:seed:local
```

The seeder creates two dummy accounts:

- `user1@example.invalid` — User One
- `user2@example.invalid` — User Two

Never run `seeds/development.sql` against production. Applied migrations are immutable; schema changes must use a new forward-only migration.

3.3 Run the Worker and frontend

First terminal:

```
npm run dev
```

Second terminal:

```
npm run dev:frontend
```

Open `http://localhost:5173`. Vite proxies `/api` to the Worker at `http://localhost:8787` and injects the dummy User One identity.

> The development environment trusts an email header to simplify testing. Never expose the

> development Worker to the Internet. Production must only accept a verified Cloudflare Access JWT.

Smoke test:

```
curl http://localhost:8787/api/health
curl -H "Cf-Access-Authenticated-User-Email: user1@example.invalid" \
  http://localhost:8787/api/me
```

PowerShell:

```
curl.exe "http://localhost:8787/api/health"
curl.exe -H "Cf-Access-Authenticated-User-Email: user1@example.invalid" `
  "http://localhost:8787/api/me"
```

3.4 Local Web Push (optional)

Generate one VAPID key pair and store it securely:

```
npx --yes web-push generate-vapid-keys --json
```

Copy `.dev.vars.example` to `.dev.vars`, then set:

```
APP_ENV="development"
WEB_PUSH_VAPID_SUBJECT="mailto:contact@example.com"
WEB_PUSH_VAPID_PUBLIC_KEY="REPLACE_WITH_PUBLIC_KEY"
WEB_PUSH_VAPID_PRIVATE_KEY="REPLACE_WITH_PRIVATE_KEY"
```

The public key may be sent to browsers. The private key must only exist in `.dev.vars` or a Cloudflare Secret. Rotating the key requires devices to subscribe again. Web Push works on `localhost`; on iOS/iPadOS, install the application to the Home Screen first.

3.5 Local Receipt Scan (optional)

The `AI` binding is already declared in `wrangler.jsonc`, but Workers AI still consumes a remote resource during `wrangler dev`. Sign in to Cloudflare:

```
npx wrangler login
npx wrangler whoami
```

The application uses `@cf/meta/llama-3.2-11b-vision-instruct`. Accept the model license through the target account's Workers AI dashboard before first use. Check the latest Cloudflare pricing and free allocation because they can change.

The browser resizes the image and strips EXIF before sending it to AI. The image is not persisted to D1, R2, KV, Cache, or IndexedDB. Scan output is only a draft and still requires user confirmation.

3.6 Quality gate

```
npm run format:check
npm run lint
npm run typecheck
npm test
npm run build
```

4. Cloudflare production setup

4.1 Sign in and define resources

```
npx wrangler login
npx wrangler whoami
```

Choose development/production Worker names, development/production D1 names, the application hostname, and two user email addresses. Do not reuse identifiers from the official Rangkumin installation for a buyer's instance.

4.2 Create D1 databases

```
npx wrangler d1 create DEVELOPMENT_DATABASE_NAME
npx wrangler d1 create PRODUCTION_DATABASE_NAME
```

Add each command's `database_name` and `database_id` to `wrangler.jsonc`:

- use the top-level binding for development;
- use `env.production.d1_databases` for production;
- keep the binding name as `DB`.

Also replace:

- the top-level Worker `name`;
- `env.production.name`;
- the hostname under `env.production.routes`;
- the VAPID subject fallback domain in `src/index.ts` and `src/routes/transactions.ts` if needed.

Keep `workers_dev: false` and `preview_urls: false` in production so no alternate origin bypasses Access.

4.3 Apply migrations

```
npm run db:migrations:list:production
npm run db:migrate:production
```

```
npm run db:migrations:list:production
```

The `db:provision:production` script does not create D1 and does not apply migrations. Create and migrate the database first.

4.4 Provision two users

```
cp config/users.production.example.json config/users.production.local.json
```

PowerShell:

```
Copy-Item "config/users.production.example.json" "config/users.production.local.json"
```

Set exactly two users with IDs `user-1` and `user-2`, distinct Cloudflare Access email addresses, and display names between 1 and 80 characters.

Validate, then apply:

```
npm run db:provision:production
npm run db:provision:production -- --apply
```

4.5 Protect the hostname with Cloudflare Access

Before exposing the production hostname:

1. Open **Cloudflare Zero Trust** → **Access controls** → **Applications**.
2. Create a **Self-hosted application** for the application hostname.
3. Add an `Allow` policy for only the two production email addresses.
4. Select an identity provider, such as One-time PIN or Google.
5. Copy the **Application Audience (AUD) Tag** and team domain `https://teamname.cloudflareaccess.com`.

Copy the config:

```
cp config/access.production.example.json config/access.production.local.json
```

Set `ACCESS_TEAM_DOMAIN` and `ACCESS_AUD`, then validate:

```
npm run access:secrets:production
```

The edge Access policy and the D1 user mapping must use the same email addresses. A user who passes Access but is not active in D1 receives `403`.

4.6 Configure production VAPID

```
npx --yes web-push generate-vapid-keys --json
cp config/web-push.production.example.json config/web-push.production.local.json
```

Set the `mailto:` subject, public key, and private key. Validate:

```
npm run webpush:secrets:production
```

Never commit `config/*.local.json`. Keep a private-key backup in a secure secret manager.

4.7 Accept the Workers AI license

In Cloudflare Dashboard, open Workers AI, select `@cf/meta/llama-3.2-11b-vision-instruct`, and accept the Meta license for the target account. Avoid placing literal API tokens in shell history. Workers AI may incur charges; always check current official pricing before selling a deployment package.

4.8 Build and perform the initial deployment

```
npm run format:check
npm run lint
npm run typecheck
npm test
npm run build
npx wrangler deploy --env production
```

After the Worker exists, apply secrets:

```
npm run access:secrets:production -- --apply
npm run webpush:secrets:production -- --apply
```

List secret names without printing values:

```
npx wrangler secret list --env production
```

Expected secrets:

- `ACCESS_TEAM_DOMAIN`
- `ACCESS_AUD`
- `WEB_PUSH_VAPID_SUBJECT`
- `WEB_PUSH_VAPID_PUBLIC_KEY`
- `WEB_PUSH_VAPID_PRIVATE_KEY`

4.9 Verify the custom domain and scheduler

Deployment installs the custom domain and these cron schedules:

- `*/15 * * * *` — reminders, budgets, fan-out, and Web Push delivery;
- `15 17 * * *` — regular processing plus daily Trash purge (17:15 UTC / approximately 00:15 WIB).

Check **Workers & Pages** → **Worker** → **Triggers** and monitor logs:

```
npx wrangler tail --env production
```

5. Production smoke-test checklist

- The hostname redirects unauthenticated visitors to Cloudflare Access.
- Both users can sign in and `GET /api/me` returns the correct identity.
- A partner can read transactions but cannot mutate another user's personal transaction.
- A new income/expense sends dashboard and Web Push notifications to the partner.
- Receipt Scan only fills a draft and does not persist a transaction before confirmation.
- Reminder and budget alerts run through cron without duplicates.
- CSV/Excel export and import are tested using dummy data.
- The PWA installs, offline reload shows a snapshot, and an offline transaction syncs exactly once after reconnecting.
- Web Push is tested on a physical device; iOS is tested from the Home Screen application.
- Security headers include CSP `blob:` for preview and `camera=(self)`.

6. Maintenance and backup

```
npm run db:migrations:list:production
npx wrangler d1 time-travel info DB --env production
npm audit
```

Never run a restore without approval and the correct bookmark. See `docs/database-operations.md` for database operations.

7. PDF documentation

Regenerate the Indonesian and English PDFs:

```
npm run docs:pdf
```

Technical guide output:

- `docs/pdf/rangkumin-setup-id.pdf`
- `docs/pdf/rangkumin-setup-en.pdf`

Step-by-step beginner guide output:

- `docs/pdf/rangkumin-installation-beginner-id.pdf`
- `docs/pdf/rangkumin-installation-beginner-en.pdf`

Set `CHROME_PATH` or `EDGE_PATH` if a Chromium browser is not detected automatically.

8. License and commercial services

The application core is MIT-licensed. Setup, support, custom branding, managed service, or private add-ons may be sold under a separate agreement. Read `LICENSE` and `COMMERCIAL.md`; obtain legal review before using final commercial terms.